

9 Ways to Do Website Security Testing & Critical Best Practices

What Is Website Security Testing?

Website security testing is the process of evaluating a website or web application's security measures to identify potential vulnerabilities, weaknesses, or flaws that could be exploited by attackers. The goal of website security testing is to ensure the confidentiality, integrity, and availability of the site, protect sensitive data, and maintain the trust of users.

Regular website security testing is crucial to maintaining a secure online presence and protecting sensitive data from being compromised. It helps organizations identify and fix security vulnerabilities, comply with industry regulations and standards, and maintain user trust.

How Can You Conduct Web Security Testing?

There are two main approaches to conducting web security testing: manual testing and automated testing. Both approaches have their advantages and limitations and are often used in combination to achieve comprehensive testing coverage.

Manual Testing

Manual testing involves the use of human expertise and intuition to identify vulnerabilities that may be missed by automated tools. The tester interacts with the website as a user, attempting to exploit vulnerabilities by manipulating input fields, cookies, and HTTP requests.

Manual testing requires a deep understanding of web application security and is time-consuming and labor-intensive. However, it can uncover complex vulnerabilities that automated tools may not detect.

Automated Testing

Automated testing involves the use of software tools to scan the website for vulnerabilities and weaknesses automatically. Automated tools can quickly identify common vulnerabilities and reduce the time and effort required for testing. However, they may generate false positives or miss complex vulnerabilities that require manual testing.

9 Website Security Testing Techniques and Tools

Website security testing techniques are various methods used to evaluate the security of a website or web application. These techniques help identify vulnerabilities, weaknesses, and flaws that could be exploited by attackers. Some common website security testing techniques and tools include:

1. **Vulnerability scanning:** Automated tools are used to scan websites for known vulnerabilities, misconfigurations, or outdated components. Vulnerability scanners can quickly identify security issues and provide remediation advice. *Learn more in our detailed guide to website security scan (coming soon)*
2. **Penetration testing:** In this manual testing methodology, security professionals simulate real-world attacks on a website or web application to identify vulnerabilities and weaknesses that automated tools may miss. Penetration testing typically involves a combination of vulnerability exploitation, social engineering, and other attack techniques.
3. **Code review:** Developers or security experts manually review the source code of a web application, looking for potential security flaws, vulnerabilities, or areas for improvement. Code reviews can help ensure that best practices are followed and that proper input validation, error handling, and encryption mechanisms are in place.
4. **Fuzz testing:** This technique involves providing unexpected, malformed, or random input data to a web application to test its resilience and identify vulnerabilities or crashes. Fuzz testing can help uncover issues like buffer overflows, memory leaks, or input validation vulnerabilities.
5. **Configuration review:** Security experts evaluate the configuration settings of web servers, application servers, databases, and other components of the technology stack to ensure they are properly secured and up-to-date.
6. **Business logic testing:** This technique focuses on analyzing the application's business logic to identify potential flaws or vulnerabilities that could lead to abuse or unauthorized actions. Business logic testing typically involves manual analysis and the use of custom test cases.
7. **API testing:** If the web application uses APIs, security testing should also include evaluating the APIs for vulnerabilities like insecure data exposure, weak authentication, and access control flaws.
8. **Static Application Security Testing (SAST):** SAST tools analyze the source code of a web application to identify potential vulnerabilities, coding issues, or insecure

programming practices. Examples of SAST tools include SonarQube, Checkmarx, and Fortify.

9. **Dynamic Application Security Testing (DAST):** DAST tools interact with a running web application, probing for security issues like input validation vulnerabilities, authentication flaws, and more. Burp Suite and OWASP ZAP are popular DAST tools that can be used for this purpose.

Website Security Testing Best Practices

Prioritize Cross Browser Compatibility Testing

Cross-browser compatibility testing is essential for website security testing because different browsers may interpret the website's code differently, which can lead to security issues. For example, a website that is secure on one browser may have security vulnerabilities on another.

By testing the website on multiple browsers and versions, you can identify and fix any vulnerabilities that may exist. It's also important to test the website on mobile devices, as they use different browsers and may have different security issues.

Adopt Risk-Based Testing

Risk-based testing focuses on identifying and addressing the most critical vulnerabilities and threats. This approach involves assessing the likelihood and potential impact of each vulnerability to prioritize testing efforts, ensuring that the most significant risks are addressed first.

To effectively implement risk-based testing, organizations should establish a clear understanding of their website's assets and the potential threats they face. This includes identifying sensitive data, such as user credentials, payment information, or personal data, as well as understanding the types of attacks that are most likely to target the website.

Once potential risks have been identified, organizations should prioritize testing efforts based on the severity of each vulnerability and its potential impact on the website's security. High-risk vulnerabilities that could lead to significant data breaches or disruptions in service should be addressed first, followed by lower-risk issues.

Risk-based testing should be an ongoing process, with regular reassessments of the website's security posture and threat landscape. This approach ensures that organizations are continually aware of emerging threats and vulnerabilities and can quickly adapt their testing efforts to address new risks.

Create a Bug Bounty Program

A bounty program is a reward-based system that encourages ethical hackers to identify and report security vulnerabilities to the website owner or developer. Here are some ways in which a bounty program can be useful for website security testing:

- **Identifying security vulnerabilities:** Ethical hackers participating in the bounty program can use their knowledge and skills to identify and report vulnerabilities that may be difficult to detect using automated tools or manual testing.
- **Building a positive reputation:** A bounty program can help build a positive reputation for the website owner or developer. It demonstrates a commitment to website security and a willingness to work with the security community to identify and fix vulnerabilities.
- **Reducing the cost of security testing:** A bounty program can help reduce the cost of website security testing. Instead of hiring security experts to conduct regular security testing, a bounty program can encourage ethical hackers to do the testing for a reward.

Practice Penetration Testing

Penetration testing, also known as ethical hacking or pen testing, is a critical best practice for website security testing. It involves simulating real-world cyberattacks on your website or web application to identify vulnerabilities and weaknesses before malicious hackers can exploit them. By conducting regular penetration tests, you can uncover security issues that may have been missed by automated tools or manual testing and take appropriate steps to address them.

Here's how you can effectively practice penetration testing for your website security testing:

1. **Set objectives:** Define goals and scope of the testing, determining which parts of your website need testing and types of attacks to simulate.
2. **Choose methodology:** Select the appropriate methodology (e.g., black-box, white-box, gray-box) based on your needs and resources.
3. **Engage skilled testers:** Hire or train experienced penetration testers with necessary skills and certifications.
4. **Combine manual and automated testing:** Use both techniques for comprehensive coverage, identifying common and complex vulnerabilities.
5. **Follow a structured approach:** Adopt a systematic framework like OWASP Testing Guide or PTES for identifying, exploiting, and reporting vulnerabilities.
6. **Document and remediate:** Record identified vulnerabilities, their severity, impact, and suggested remediation steps. Prioritize and address them based on risk level.
7. **Conduct regular tests:** Perform penetration tests regularly, especially after significant website changes, to ensure up-to-date and effective security measures.

Website Security Testing with HackerOne

The HackerOne Attack Resistance Platform helps your organization anticipate threats with adversarial testing by ethical hackers who work for you. They continuously discover and prioritize vulnerabilities in your internet applications and can enrich your current website security testing processes with findings prioritized by risk. The Attack Resistance Platform innovates your security faster than cybercrime so you can evolve your digital business with confidence.

[Learn more about the HackerOne Attack Resistance Platform](#)

Company

Leadership

Careers

Partners

Newsroom

Contact Us

Knowledge

Center

Application Security

Penetration Testing

Cloud Security

Hacking

Cybersecurity

Attacks

DevSecOps

Resources

Blog

Documentation

Leaderboard

Partner Portal

Resources

Contacted
by a  hacker?

[Cookie Preferences](#)

[Policies](#)

[Terms](#)

[Privacy](#)

[Security](#)

©2025 HackerOne All rights reserved.

Trust